Guia explicativa paso a paso

Construccion del Dashboard Ejecutivo de Bajas TI

Universidad Autonoma de Occidente - Unidad de Servicio al Usuario

Este documento describe, paso a paso, como se construyo la aplicacion web (Dashboard Ejecutivo) a partir de las solicitudes de baja de activos (DITI), explica el script de Python empleado para transformar los datos y detalla las herramientas de terceros utilizadas.

Contenido

- 1. Vision general del flujo de trabajo
- 2. Paso 1 - Lectura y comprension de los datos de origen
- 3. Paso 2 - Normalizacion con Python (script explicado)
- 4. Paso 3 - Diseno de la aplicacion HTML (KPIs, graficos, filtros)
- 5. Paso 4 - Validacion de los datos
- 6. Paso 5 - Publicacion en GitHub y GitHub Pages
- 7. Herramientas de terceros utilizadas
- 8. Resumen de resultados

1. Vision general del flujo

El objetivo fue convertir un listado tabular de bajas de equipos de computo en un tablero ejecutivo interactivo, autocontenido en un unico archivo HTML. El flujo siguio cinco etapas:

- Datos de origen: tabla con columnas Archivo de Origen, Fecha, Consecutivo (DITI), Cantidad, Descripcion del elemento, Serie No. y No. inventario.
- Normalizacion: un script de Python clasifica cada fila por categoria y marca, convierte las fechas y genera un arreglo de datos listo para el navegador.
- Interfaz: HTML + CSS + JavaScript (Chart.js) con KPIs, 5 graficos, filtros y modo claro/oscuro, todo embebido en un solo archivo.
- Validacion: se verifican totales (lineas, unidades, documentos) y la sintaxis.
- Publicacion: el archivo se sube a un repositorio GitHub y se expone con Pages.

2. Paso 1 - Lectura de los datos de origen

La fuente final fue un archivo CSV exportado desde Excel. Cada fila representa un activo individual dado de baja. Un ejemplo de las primeras lineas:

```
Archivo de Origen,Fecha,Consecutivo,Cantidad,Descripcion del elemento,Serie No.,No. inventario
Bajas 2025.pdf,16/1/2025,DITI-8.6-01,1,TELEFONO IP GAMA MEDIA 1608,10WZ25052647,98975
Bajas 2025.pdf,20/1/2025,DITI-8.6-03,1,Teclado Logitech EX100,820-001810,41940
Bajas 2025.pdf,21/4/2025,DITI-4.6-23,1,"Imac 21""",QP05103PDB6,100033
```

Observaciones clave que condicionaron el procesamiento:

- Las descripciones con comas van entrecomilladas (CSV estandar).
- La columna Cantidad puede ser mayor que 1 (p. ej. lotes de mouse).
- Algunos campos de serie o inventario vienen vacios.
- Las fechas estan en formato d/m/aaaa (debe convertirse a aaaa-mm-dd).
- Hay dos archivos de origen: Bajas 2025.pdf y bajas 2026.pdf.

3. Paso 2 - Normalizacion con Python

Se escribio un script en Python (solo libreria estandar: modulos csv y json) que realiza cuatro tareas: (a) leer el CSV, (b) convertir la fecha, (c) clasificar cada fila en una categoria y una marca mediante palabras clave, y (d) exportar un arreglo JavaScript (const RAW = [...]) que el dashboard consume directamente.

3.1 Lectura del CSV

Se usa csv.DictReader para acceder a cada columna por su nombre de encabezado:

```
import csv, json

rows = []
with open('bajas.csv', encoding='utf-8') as f:
    r = csv.DictReader(f)          # lee usando la fila de encabezados
    for row in r:
        rows.append(row)           # cada 'row' es un diccionario
```

3.2 Conversion de la fecha

La fecha d/m/aaaa se normaliza a formato ISO aaaa-mm-dd, que permite ordenar y agrupar por mes de forma fiable:

```
def parse_date(d):
    dd, mm, yy = d.strip().split('/')
    return f'{int(yy):04d}-{int(mm):02d}-{int(dd):02d}'

# Ejemplo:  '16/1/2025'  ->  '2025-01-16'
```

3.3 Clasificacion por categoria

Como la columna 'Descripcion' es texto libre, se definio una funcion que asigna una categoria normalizada buscando palabras clave en minusculas. El orden de las reglas importa (se evalua celular y portatil antes que CPU para evitar choques):

```
def categorize(desc):
    d = desc.lower()
    if any(k in d for k in ['celular','smartphone','galaxy',
                            'alcatel','nokia','moto e','motorola']):
        return 'Celular'
    if ('portatil' in d or 'macbook' in d or 'notebook' in d
            or 'latitude' in d):
        return 'Portatil'
    if any(k in d for k in ['cpu','computador','optiplex','imac',
                            'macpro','desktop','todo en uno','aio']):
        return 'CPU/Computador'
    if 'monitor' in d or 'pantalla' in d or 'display' in d:
        return 'Monitor'
    if 'teclado' in d or 'keyboard' in d:   return 'Teclado'
    if 'mouse' in d:                         return 'Mouse'
    if 'diadema' in d or 'parlante' in d:    return 'Diadema/Audio'
    if 'telefono' in d or 'avaya' in d:      return 'Telefono'
    if 'ipad' in d or 'tablet' in d or 'wacom' in d: return 'Tablet'
```

```
    if any(k in d for k in ['cisco','switch','enrutador','servidor']):
        return 'Red/Networking'
    return 'Otros'                          # categoria por defecto
```

3.4 Deteccion de la marca

De forma analoga, una lista de pares (palabra_clave, marca) permite inferir el fabricante. El primer patron que coincide gana; si ninguno coincide, la marca es 'Generico':

```
def brand(desc):
    d = desc.lower()
    pairs = [('dell','Dell'), ('optiplex','Dell'), ('imac','Apple'),
             ('macbook','Apple'), ('hp ','HP'), ('avaya','Avaya'),
             ('yealink','Yealink'), ('samsung','Samsung'),
             ('cisco','Cisco'), ('sennheiser','Sennheiser'),
             ('plantronics','Plantronics'), ('logitech','Logitech'),
             ('xkim','Xkim'), ('lenovo','Lenovo'), ('lg ','LG')]
    for clave, marca in pairs:
        if clave in d:
            return marca
    return 'Generico'
```

3.5 Exportacion del arreglo RAW

Finalmente se recorre cada fila, se calculan categoria y marca, se acumulan los totales de control y se escribe el arreglo JavaScript que el HTML incluye tal cual:

```
out = []
for row in rows:
    desc = row['Descripcion del elemento'].strip()
    if not desc:            # ignora filas sin descripcion
        continue
    qty = int(row['Cantidad'] or '1')
    out.append({
        'src':   row['Archivo de Origen'].strip(),
        'doc':   row['Consecutivo'].strip(),
        'date':  parse_date(row['Fecha']),
        'qty':   qty,
        'cat':   categorize(desc),
        'brand': brand(desc),
        'desc':  desc,
        'serie': row['Serie No.'].strip(),
        'inv':   row['No. inventario'].strip(),
    })

# Totales de control (se imprimen para validar):
print('Lineas :', len(out))
print('Unidades:', sum(o['qty'] for o in out))
print('Docs   :', len({o['doc'] for o in out}))

# Se escribe el arreglo que consumira el dashboard:
with open('_raw.js','w',encoding='utf-8') as f:
    f.write('const RAW = [\n')
    for o in out:
        f.write('  ' + json.dumps(o, ensure_ascii=False) + ',\n')
    f.write('];\n')
```

El resultado es un objeto por activo, por ejemplo:

```
{"src":"Bajas 2025.pdf","doc":"DITI-8.6-01","date":"2025-01-16",
 "qty":1,"cat":"Telefono","brand":"Avaya",
 "desc":"TELEFONO IP GAMA MEDIA 1608","serie":"10WZ25052647",
 "inv":"98975"}
```

4. Paso 3 - Diseno de la aplicacion HTML

El dashboard es un unico archivo .html que incluye CSS y JavaScript embebidos. Solo depende de una libreria externa (Chart.js), cargada por CDN. Sus bloques:

4.1 KPIs (tarjetas superiores)

Seis tarjetas calculan metricas agregadas en tiempo real: unidades dadas de baja, numero de solicitudes DITI, categorias, participacion de la categoria principal, porcentaje de equipos de computo y numero de marcas.

4.2 Graficos interactivos (Chart.js)

- Dona: distribucion de unidades por categoria.
- Barras horizontales: participacion por marca (top 10).
- Linea + barra combinadas: tendencia mensual (unidades vs. solicitudes).
- Barras: top 8 descripciones/modelos.
- Barras: unidades por documento DITI.

4.3 Filtros avanzados

Listas desplegables (archivo de origen, mes, marca, documento), un buscador de texto y chips por categoria. Cualquier cambio vuelve a calcular KPIs, graficos y tabla sin recargar la pagina, mediante una funcion getFiltered() que filtra el arreglo RAW y un render() que repinta todo.

4.4 Diseno y modo claro/oscuro

Interfaz minimalista y responsiva. Un boton alterna el atributo data-theme del body; las variables CSS (:root) cambian la paleta y los graficos se redibujan para ajustar sus colores al tema activo.

5. Paso 4 - Validacion de los datos

Antes de publicar, se verificaron los totales y la sintaxis con Node.js, extrayendo el arreglo RAW del HTML y comprobando que no hubiera registros incompletos:

```
const fs = require('fs');
const html = fs.readFileSync('dashboard-bajas-uao.html','utf8');
const m = html.match(/const RAW = \[([\s\S]*?)\n\];/);
const RAW = eval('[' + m[1] + ']');
console.log('Registros:', RAW.length);
console.log('Unidades :', RAW.reduce((s,r)=>s+r.qty,0));
console.log('Documentos:', new Set(RAW.map(r=>r.doc)).size);
```

6. Paso 5 - Publicacion en GitHub y Pages

El archivo se versiono con Git y se subio a un repositorio de GitHub. Los comandos esenciales fueron:

```
git init
git add dashboard-bajas-uao.html README.md bajas.csv
git commit -m 'Dashboard Ejecutivo de Bajas TI'
git push -u origin main
```

Para exponer el dashboard como sitio web (GitHub Pages) se activa, en el repositorio, la

opcion Settings > Pages, eligiendo como fuente la rama main y la carpeta raiz. El dashboard queda accesible en una URL del tipo:

```
https://<usuario>.github.io/<repositorio>/dashboard-bajas-uao.html
```

7. Herramientas de terceros utilizadas

El proyecto se apoya en muy pocas dependencias externas; la mayor parte usa tecnologias estandar:

- Chart.js (v4, via CDN jsdelivr): unica libreria de terceros en la aplicacion; renderiza los cinco graficos interactivos.
- Python 3 (libreria estandar: modulos csv y json): normalizacion de datos. NO se usaron librerias de terceros de Python (ni pandas ni openpyxl).
- Node.js (incorporado en el entorno): solo para validar totales y sintaxis.
- Git y GitHub: control de versiones, alojamiento y publicacion con Pages.
- Para ESTE PDF: generador propio escrito en Python puro (libreria estandar), sin pandoc, wkhtmltopdf, reportlab ni fpdf; se escribe directamente la estructura del archivo PDF 1.4.

Tecnologias base del dashboard (sin instalacion): HTML5, CSS3 (variables, grid, flexbox) y JavaScript (ES6).

8. Resumen de resultados

- 935 lineas de activo procesadas.
- 1.338 unidades totales dadas de baja.
- 106 solicitudes (documentos DITI).
- 11 categorias y 41 marcas detectadas automaticamente.
- 2 archivos de origen: Bajas 2025.pdf y bajas 2026.pdf.
- Periodo cubierto: enero 2025 a octubre 2026.

El dashboard resultante es portatil (un solo archivo), no requiere servidor y funciona en cualquier navegador moderno. Toda la logica de agregacion ocurre en el navegador a partir del arreglo RAW generado por el script de Python.